

NAIROBI LOCAL BUS & MATATU BOOKING, TRACKING & PAYMENT APP

Software Project Documentation / Developer Brief

Project focus: Local public transport within Nairobi and surrounding areas.

Core concept: A platform similar in convenience to Uber, but designed for shared buses and matatus operating on established local routes.

1. Project Overview

The application will allow passengers to find local buses/matatus, identify routes and nearby boarding points, view active vehicles on a map, reserve a seat where supported, pay digitally, receive a digital ticket and track the vehicle in real time.

Example routes may include CBD → Rongai, CBD → Ngong, CBD → Kasarani, CBD → Thika, CBD → Kiambu, CBD → Ruaka, CBD → Kikuyu and other Nairobi-area routes.

2. Main Users

Passenger: Searches routes, finds vehicles, selects boarding and destination points, books where supported, pays, receives a ticket and tracks the vehicle.

Driver/Conductor: Uses a mobile application for vehicle assignment, route status, GPS tracking, passenger verification and QR-ticket scanning.

Transport Operator: Manages vehicles, drivers, routes, fares, bookings and revenue.

Platform Administrator: Manages the complete system, operators, vehicles, routes, payments, users and monitoring.

3. Passenger Journey

Open App → Enter Destination → Find Nearby Vehicles → Select Route/Vehicle → Select Boarding Point → Select Seat (if available) → Pay → Receive Ticket → Track Vehicle → Board.

4. Local Route System

Unlike long-distance buses, local vehicles may operate continuously. The system should therefore support routes, stops/boarding points and active vehicles rather than relying only on fixed departure times.

Example: CBD → Railways → Nyayo Stadium → Wilson → Lang'ata → Karen → Rongai.

5. Finding Nearby Vehicles

Use the passenger's approximate GPS location to show suitable vehicles currently operating nearby.

Example:

Route 125 | CBD → Rongai | Bus KDA 123A | 650 m away | ETA 4 min | Fare KSh 100

6. Real-Time Vehicle Tracking

Each participating vehicle should use a driver/conductor smartphone application. The phone's GPS sends location updates to the backend. The passenger application receives the latest position and displays the vehicle on a map.

Flow: Driver Phone → GPS → Internet → Backend → Real-Time Location → Passenger App.

Show current location, route/direction, distance from passenger, estimated arrival time, vehicle registration number and operator information.

7. Boarding Points

The app should support approved/designated boarding points along each route. A passenger can select a convenient point and receive an ETA for the selected vehicle.

8. Seat Booking

Where a vehicle supports reserved seating, passengers should see a visual seat layout and select an available seat. Booked seats must be unavailable to other passengers.

Important: Nairobi local transport does not always use advance seat reservations. The system should support both **reserved-seat mode** and **pay/board mode** without guaranteeing a specific seat.

9. Fare Management

Fares should be configurable by route, boarding point, destination, operator and, where required, time/day.

Example: Kencom → Rongai = KSh 100. Operators/admins should be able to update approved fares.

10. Online Payment

The initial payment method should support M-Pesa. Recommended flow: Select route → vehicle → destination → seat if applicable → confirm fare → M-Pesa STK Push → payment verification → booking confirmation → digital ticket.

Payment must only be marked successful after server-side confirmation from the payment provider.

11. Digital Ticket & QR Code

After successful payment, generate a digital ticket containing passenger name, route, boarding point, destination, vehicle, seat where applicable, fare, booking ID, QR code and payment status.

The conductor should be able to scan the QR code and verify the booking.

12. Driver/Conductor App

Features should include login, assigned vehicle, assigned route, start/stop route, GPS tracking, passenger list and QR scanner.

When the route starts, the driver taps START ROUTE and location tracking begins.

13. Passenger Tracking Screen

The map should resemble a ride-hailing tracking interface: passenger location, vehicle location, route and destination. ETA should be calculated using current position, distance and available traffic information.

14. Notifications

Booking confirmation, payment confirmation, seat assignment, vehicle approaching, vehicle arrival, route changes and other important trip notifications.

15. Operator Dashboard

Manage vehicles, registration numbers, capacity, drivers, routes, boarding points, fares, bookings and revenue. Revenue reports should include daily, weekly and monthly totals.

16. Administrator Dashboard

Monitor active vehicles, active drivers, bookings, payments, routes and operators. The administrator should be able to approve operators, register vehicles/drivers, manage fares, monitor vehicles on a live map, suspend accounts and generate reports.

17. How This Differs From Uber

Uber generally connects one passenger to one private vehicle. This platform connects many passengers to one shared vehicle operating on a route.

Example: Passenger A + Passenger B + Passenger C + Passenger D → Bus KDA 123A → CBD → Rongai.

The backend must therefore support multiple passengers, bookings and potentially seats for the same vehicle and route.

18. Capacity & Seat-Locking

The system must know each vehicle's capacity. Example: capacity 45, booked 32, available 13.

When a passenger selects a seat, temporarily lock it during payment (for example 5–10 minutes). If payment succeeds, mark it booked. If payment expires, release the seat.

19. Recommended Technology Stack

Mobile: Flutter or React Native.

Backend: Laravel/PHP, Node.js or Python.

Database: MySQL or PostgreSQL.

Maps: Google Maps Platform or Mapbox.

Real-time: WebSockets, Firebase or Socket.IO.

Payments: M-Pesa/Daraja integration.

Notifications: Firebase Cloud Messaging and/or SMS gateway.

20. Suggested Database Tables

users, operators, drivers, vehicles, vehicle_seats, routes, route_stops, vehicle_routes, bookings, booking_seats, payments, vehicle_locations, tickets, notifications, fares and reviews.

21. Security Requirements

Secure authentication, password hashing, role-based access, HTTPS, secure APIs, payment verification, GPS authentication, protection against duplicate bookings, seat locking, transaction logging and admin audit logs.

22. MVP — First Version

Passenger: Registration/login, route search, boarding point, destination, nearby active vehicles, live location, seat availability, booking, M-Pesa payment, digital ticket and QR code.

Driver/Conductor: Login, assigned vehicle/route, start/stop trip, GPS tracking, passenger list and QR scanner.

Admin: Operators, vehicles, drivers, routes, stops, bookings, payments, live vehicle map and reports.

23. Future Features

Multiple transport operators, ratings/reviews, trip history, favorite routes, push notifications, traffic-based ETA, customer support, lost-and-found reporting, driver performance monitoring, automated revenue sharing, wallet, promo codes, corporate accounts and commuter passes.

24. Final Product Objective

The final product should become a digital platform for Nairobi's local public transport, making it easier for passengers to find vehicles, know where to board, pay digitally, reserve seats where supported and track their vehicle in real time.

The architecture should support multiple routes, vehicles, operators, drivers, conductors and large numbers of passengers, with the ability to expand to other Kenyan towns and cities.

Suggested MVP priority: Real-time vehicle discovery and ETA should be the core feature. Seat reservation and payment should support both vehicles that allow advance reservations and vehicles that operate on a pay/board model.